LLM4ChipDesign

Enhanced LLM-Aided Testbench Generation

Report — golden models, self-checking testbenches, and a bug that got caught

---

Student	Sahil Dunani (sd6429)
Course	MS Project: Generative AI based Chip Design
Instructor	Prof. Ramesh Karri (with Dr. Weihua Xiao)
Model used	gpt-4o-mini (OpenAI API)
Simulation	Google Colab + Icarus Verilog (iverilog / vvp)
Date	October 2026

The short version: Two tutorial examples (mux and adder) and one custom module (saturating adder) all run end-to-end through the LLM-driven pipeline. All three generated self-checking testbenches pass all 20 patterns. The bug-detection demo intentionally breaks the saturating adder and the testbench catches 8 of 20 tests failing; restoring the DUT gives all 20 passing again.

Introduction

The pipeline, briefly

Everything in this submission comes from the same four-step pipeline, applied to three different modules. The pipeline is wrapped in a single `run_pipeline(name, spec, module_name, port_decl, dut_v)` helper so each example is just two cells in the notebook: define the spec + DUT, then call the pipeline. The four steps are the ones named in the assignment PDF.

- **Step 1 - initial testbench.** The LLM reads the natural-language spec and the module port declaration, and returns a Verilog testbench that applies about 20 diverse stimulus patterns (corner cases + randomized) with `$display` prints after each pattern. Alongside the Verilog it returns the exact pattern list as a JSON block, which keeps step 3 deterministic (we never have to parse the generated Verilog back out to recover the patterns).
- **Step 2 - Python golden model.** A separate LLM call reads the same spec and returns a Python file with a single function `oracle(inputs: dict) -> dict`. The function implements the specified behavior in plain Python and is written to `golden_model.py`.
- **Step 3 - run patterns through the oracle.** Python then imports `golden_model.py`, calls `oracle()` on every pattern from step 1, and writes `test_patterns_with_golden.json` - a list of {inputs, expected} dicts.
- **Step 4 - self-checking testbench.** A third LLM call is given the initial testbench and the patterns-with-golden JSON, and returns `testbench_final.v`: the same patterns in the same order, but now with hardcoded expected values, per-test PASS/FAIL `$display` lines, an `integer errors` counter, and a final `ALL TESTS PASSED` or `N TEST(S) FAILED` summary.

Verification is a plain iverilog + vvp invocation. For every example the exact command is:

```
iverilog -g2012 -o tb.vvp <module>.v testbench_final.v && vvp tb.vvp
```

LLM setup

- **Model:** `gpt-4o-mini`
- **Temperature:** 0.3 by default (lower for retries)
- **Max tokens:** 2500 (bumped to 4000 for one retry prompt that includes a long JSON block)
- **Prompt choices that mattered:** In step 1 I asked the LLM to emit the stimulus TB AND a JSON list of patterns in the same response (two separate fenced blocks), which avoids any need to parse the Verilog back out. In step 4 I had to iterate the prompt once or twice to pin the output to plain Verilog-2001 - the baseline wording let the LLM reach for SystemVerilog `++` and comma-chained statements that iverilog rejects. Full detail in the mux/adder notes below.

Part I

Two tutorial examples

Results

Example	DUT ports	Patterns	Pipeline step 4 result	Final result
mux	a, b, sel (1-bit each) -> y (1-bit)	20	LLM used SV `errors++` first; retried with stricter prompt	ALL 20 PASS
adder	a, b (4-bit), cin (1-bit) -> sum (4-bit), cout (1-bit)	20	Needed 2 retries - final used a worked-template prompt	ALL 20 PASS
sat_adder (Part III)	a, b (4-bit) -> sum (4-bit)	20	Compiled and passed on first attempt	ALL 20 PASS

Example 1 — 2-to-1 Multiplexer

A combinational 1-bit 2-to-1 mux. Spec: when sel=0, y=a; when sel=1, y=b. The DUT I provided is the one-line `assign y = sel ? b : a;`. The pipeline produced a stimulus-only initial testbench with 20 patterns, a Python oracle that returns `{'y': a if sel == 0 else b}`, and a final testbench that compares DUT output `y` to the expected value for each of the 20 patterns.

Verification command: `iverilog -g2012 -o tb.vvp mux21.v testbench_final.v && vvp tb.vvp`

Final line: **`ALL TESTS PASSED`** after 20 individual PASS lines.

Example 2 — 4-bit Adder with carry-in and carry-out

A combinational 4-bit ripple-carry adder. Spec: `{cout, sum} = a + b + cin`. The oracle is a one-liner that computes the 5-bit integer sum then splits it into the lower 4 bits (`sum`) and bit 4 (`cout`). The initial TB applied 20 patterns including the obvious corners (0+0, 15+0, 15+15 with and without cin, etc.) plus randomized combinations.

This example is where the step-4 prompt needed the most iteration. The baseline prompt let the LLM write SystemVerilog syntax that iverilog rejects. The first retry (with a stricter rules list) produced a syntax error at a later line. The second retry — which included a fully worked TEST block in the prompt as an example — produced a clean testbench that compiled and ran. I kept all three attempts in the notebook so the trajectory is visible.

Verification command: `iverilog -g2012 -o tb.vvp adder4.v testbench_final.v && vvp tb.vvp`

Final line: **`ALL TESTS PASSED`** after 20 individual PASS lines.

Part II

Inspect and explain the generated artifacts (adder)

Golden model — `golden_model.py`

For the adder, the LLM produced this four-line oracle:

```
def oracle(inputs: dict) -> dict:
    a   = inputs['a']
    b   = inputs['b']
    cin = inputs['cin']
    total = a + b + cin
    return {'sum': total & 0xF, 'cout': (total >> 4) & 0x1}
```

The input dict is keyed by the three input port names (`a`, `b`, `cin`) with integer values. The function returns a dict keyed by the two output port names (`sum`, `cout`) with integer values. The computation is just Python integer arithmetic: build the 5-bit sum, then slice off the lower nibble and bit 4. This exactly mirrors the Verilog `{cout, sum} = a + b + cin` behavior.

Pattern file — `test_patterns_with_golden.json`

The JSON file has one top-level key `patterns` whose value is a list of 20 entries. Each entry has two fields: `inputs` (a dict mapping each input port to an integer) and `expected` (a dict mapping each output port to the integer that the oracle returned). Showing the first three entries verbatim:

```
{"inputs": {"a": 0, "b": 0, "cin": 0},  "expected": {"sum": 0,  "cout": 0}}
{"inputs": {"a": 0, "b": 0, "cin": 1},  "expected": {"sum": 1,  "cout": 0}}
{"inputs": {"a": 15, "b": 15, "cin": 0}, "expected": {"sum": 14, "cout": 1}}
```

This file is the single source of truth that step 4 uses to generate the self-checking testbench. Expected values are stored as plain integers (not Verilog literals) because the step-4 LLM formats them into Verilog constants in whatever width the DUT port requires.

Updater effect — what changed between initial and final testbench

The initial testbench applies stimulus and prints what it sees, no checking. The final testbench applies the same stimulus but adds expected values and `if (actual !== expected)` checks. Concrete before / after:

Initial testbench (one test)

```
a = 4'd15; b = 4'd15; cin = 1'b0; #1;
$display("TEST 5: a=%b b=%b cin=%b sum=%b cout=%b", a, b, cin, sum, cout);
```

Final testbench (same test)

```
a = 4'd15; b = 4'd15; cin = 1'b0; #1;
if (sum !== 4'd14 || cout !== 1'b1) begin
    $display("TEST 5 FAIL: a=15 b=15 cin=0 expected sum=14 cout=1 got sum=%0d cout=%0d", sum,
cout);
    errors = errors + 1;
end else begin
```

```
        $display("TEST 5 PASS: a=15 b=15 cin=0 sum=%0d cout=%0d", sum, cout);
    end
```

The final version has the hardcoded expected values (sum=14, cout=1), an `if` that compares the actual DUT outputs against them, a PASS branch and a FAIL branch, and an `errors = errors + 1;` counter in the FAIL branch. At the end of the test sequence the final TB prints `ALL TESTS PASSED` if errors==0, otherwise `N TEST(S) FAILED` — which is the mechanism Part IV exercises.

Part III

Custom module — 4-bit saturating adder

Design choice

I picked a 4-bit saturating adder. It has a clean small interface (two 4-bit inputs, one 4-bit output), one meaningful corner behavior (saturation when the raw sum exceeds 15), and the saturation vs wrap-around distinction is easy to describe in the spec and exercise in the testbench. It is also a natural setup for Part IV, since removing the saturation produces a subtle bug that only affects overflow cases - exactly the kind of thing a well-chosen test suite should catch.

Specification given to the pipeline

> *A 4-bit saturating adder. Inputs: a (4-bit unsigned), b (4-bit unsigned). Output: sum (4-bit unsigned). Behavior: compute the 5-bit arithmetic sum of a and b. If the 5-bit sum is less than or equal to 15, output that value in `sum`. If the 5-bit sum is greater than 15 (overflow), SATURATE the output to 4'd15 (all ones). Purely combinational, no clock, no reset.* Followed by a few concrete examples (0+0=0, 7+5=12, 8+8=15 saturating, 15+15=15 saturating).

DUT interface

```
module sat_adder4(
    input  wire [3:0] a,
    input  wire [3:0] b,
    output wire [3:0] sum
);
```

Pipeline run and verification

The pipeline ran cleanly on the first attempt - no prompt retry needed for either the golden model or the final testbench. The LLM produced 20 patterns that included every important corner (0+0, 15+0, 8+8=saturate, 15+15=saturate, several mid-range non-overflow sums, and a mix of random overflow and non-overflow combinations). The oracle returned `min(a + b, 15)`.

Verification command: `iverilog -g2012 -o tb.vvp sat_adder4.v testbench_final.v && vvp tb.vvp`

Result: **`ALL TESTS PASSED`** - all 20 patterns matched.

Part IV

Bug detection demonstration

To show the generated testbench actually catches implementation bugs, I introduced a small intentional bug into the saturating-adder DUT - specifically, I removed the saturation check so overflowing sums wrap around instead of being clipped to 15. This is the exact mistake a tired engineer might make: skipping the overflow case and letting Verilog's natural 4-bit truncation take over.

Step 1 - the buggy DUT

```
module sat_adder4(input wire [3:0] a, input wire [3:0] b, output wire [3:0] sum);
    // BUG INJECTED: no saturation. Overflow wraps instead of clipping.
    assign sum = a + b;
endmodule
```

I re-ran the same `testbench_final.v` from Part III against this buggy DUT. The expected behavior is that non-overflow patterns still pass (because `a + b` is the same thing under 15), but every overflow pattern should fail because the DUT now returns the wrapped sum instead of 15.

Step 1 result - the testbench catches it

Snippet of the simulation output (the full output is in the notebook):

```
TEST 3 PASS: a=15 b= 0 sum=15
TEST 4 FAIL: a= 8 b= 8 expected sum=15 got= 0
TEST 5 FAIL: a=15 b=15 expected sum=15 got=14
TEST 9 FAIL: a=12 b= 4 expected sum=15 got= 0
TEST 10 FAIL: a=14 b= 3 expected sum=15 got= 1
TEST 16 FAIL: a=15 b= 1 expected sum=15 got= 0
TEST 17 FAIL: a= 9 b= 7 expected sum=15 got= 0
TEST 18 FAIL: a=15 b= 2 expected sum=15 got= 1
TEST 19 FAIL: a=11 b= 5 expected sum=15 got= 0
8 TEST(S) FAILED
```

Exactly as expected: 12 patterns still pass (those are the non-overflow cases, where saturation and plain addition agree), and 8 patterns fail (those are the overflow cases, where the buggy DUT returns the wrapped lower nibble but the spec requires 15). This is the self-checking testbench doing its job - a lone print statement would not have told me anything was wrong.

Step 2 - restore the DUT and re-run

Restoring the correct DUT (reintroducing the `(raw > 5'd15) ? 4'd15 : raw[3:0]` select) and re-running the same `testbench_final.v` produces `ALL TESTS PASSED` again across all 20 patterns. Nothing about the testbench had to change - the oracle, the expected values, and the test harness were all unchanged. Only the DUT was swapped. That is the whole point of a self-checking testbench: once it is produced, it is a stable artifact you can rerun across design iterations.

Final line after fix: **`ALL TESTS PASSED`** - all 20/20 after the fix.